List of Experiments

Total No of Labs: 14

S. No.	Programs List	No. of Labs
CAD Experiments		
1	Study of basic graphics functions defined in “graphics.h”.	1
2	To implement Digital differential analyzer (DDA) Algorithm for drawing a line segment between two given end points A (x1, y1) and B(x2, y2).	1
3	To implement Bresenham’s line drawing algorithm for drawing a line segment between two given endpoints A (x1, y2) and B(x2, y2).	1
4	To implement Bresenham’s Cicrcle drawing algorithm for drawing a circle with a given center of circle P(x, y) and radius r.	1
5	To apply the basic 2D transformations such as translation, Scaling, Rotation, shearing and reflection for a given 2D object.	2
6	To apply the basic transformations such as translation, Scaling, Rotation for a given 3D object.	2
7	To generate a smooth curve by using Bezier curve technique for a given set of control points.	1
8	To implement Cohen–Sutherland 2D clipping and window–viewport mapping	1
Product Design		
9	Introduction to CAD software’s.	1
10	Generation of any one simple solid model using any CAD software.	1
11	Generation of any one assembly of mechanical part using any CAD software.	1
12	Design and develop the complete lifecycle for any product.	1

Experiment No. – 5

Aim: To apply the basic 2D transformations such as translation, Scaling, Rotation, shearing and reflection for a given 2D object.

Theory: In generalized form, the matrix equation incorporating all five types of geometric transformations may be expressed as

$$\begin{Bmatrix} x_T \\ y_T \\ 1 \end{Bmatrix} = \begin{bmatrix} A & B & 0 \\ C & D & 0 \\ 0 & 0 & 1 \end{bmatrix} . \begin{Bmatrix} x \\ y \\ 1 \end{Bmatrix} = [T] . \begin{Bmatrix} x \\ y \\ 1 \end{Bmatrix} \tag{1}$$

where $[T]$ represents a transformation matrix in homogeneous coordinates. Different 2D geometric transformation matrices in homogeneous coordinates are

Translation:

$$[T_t] = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \tag{2}$$

Rotation:

Counter clockwise rotation (ccw) in the xy plane,

$$[T_r] = \begin{bmatrix} \cos\alpha & -\sin\alpha & 0 \\ \sin\alpha & \cos\alpha & 0 \\ 0 & 0 & 1 \end{bmatrix} \tag{3}$$

Clockwise rotation (cw) in the xy plane,

$$[T_r] = \begin{bmatrix} \cos\alpha & \sin\alpha & 0 \\ -\sin\alpha & \cos\alpha & 0 \\ 0 & 0 & 1 \end{bmatrix} \tag{4}$$

Scaling:

$$[S] = \begin{bmatrix} S_x & 0 & 0 \\ 0 & S_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \tag{5}$$

Reflection:

About the x-axis,

$$[R_x] = \begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \tag{6}$$

About the y-axis,

$$[R_y] = \begin{bmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \tag{7}$$

About the origin, $$[R]=\begin{bmatrix}-1 & 0 & 0\\ 0 & -1 & 0\\ 0 & 0 & 1\end{bmatrix} \quad (8)$$

About the line $y=x$, $$[R]=\begin{bmatrix}0 & 1 & 0\\ 1 & 0 & 0\\ 0 & 0 & 1\end{bmatrix} \quad (9)$$

Shear :

Along the *x*-axis, $$[Sh_x]=\begin{bmatrix}1 & Sh_x & 0\\ 0 & 1 & 0\\ 0 & 0 & 1\end{bmatrix} \quad (10)$$

Along the *y*-axis, $$[Sh_y]=\begin{bmatrix}1 & 0 & 0\\ Sh_y & 1 & 0\\ 0 & 0 & 1\end{bmatrix} \quad (11)$$

Along the x and *y*-axes, $$[Sh]=\begin{bmatrix}1 & Sh_x & 0\\ Sh_y & 1 & 0\\ 0 & 0 & 1\end{bmatrix} \quad (12)$$

Scaling and Shear: The effect of elements A, B, C and D in 3x3 transformation matrix (eqn. 4.18) may be separately identified. The terms B and C cause shear along x and y directions (eqn. 4.23c), respectively. The terms A and D act as scale factors (eqn. 4.21). Thus, the general 3x3 transformation matrix (eqn. 4.18) producing a combination of shear and scaling, is expressed as

$$[T]=\begin{bmatrix}S_x & Sh_x & 0\\ Sh_y & S_y & 0\\ 0 & 0 & 1\end{bmatrix} \quad (13)$$

In general, transformations with a determinant identically equal to +1 gives pure rotations. However, if the determinant of matrix is identically -1 then the transformation produces pure reflection. If two pure reflection transformations applied successively about the lines passing through origin, it results into a pure rotation.

Program for translation:

```
#include<stdio.h>
#include<conio.h>
#include<graphics.h>
#include<math.h>
void main()
{
int gd=DETECT,gm;
int x1,y1,x2,y2,tx,ty,x3,y3,x4,y4;
initgraph(&gd,&gm,"C:\\TurboC3\\BGI");
```

```
printf("Enter the starting point of line segment:");
scanf("%d %d",&x1,&y1);
printf("Enter the ending point of line segment:");
scanf("%d %d",&x2,&y2);
printf("Enter translation distances tx,ty:\n");
scanf("%d%d",&tx,&ty);
setcolor(5);
line(x1,y1,x2,y2);
outtextxy(x2+2,y2+2,"Original line");
x3=x1+tx;
y3=y1+ty;
x4=x2+tx;
y4=y2+ty;
setcolor(7);
line(x3,y3,x4,y4);
outtextxy(x4+2,y4+2,"Line after translation");
getch();
}
```

Program for scaling:

```
#include<stdio.h>
#include<conio.h>
#include<graphics.h>
#include<math.h>
void main()
{
int gd=DETECT,gm;
float x1,y1,x2,y2,sx,sy,x3,y3,x4,y4;
initgraph(&gd,&gm,"C:\\TurboC3\\BGI");
printf("Enter the starting point coordinates:");
scanf("%f %f",&x1,&y1);
printf("Enter the ending point coordinates:");
scanf("%f %f",&x2,&y2);
printf("Enter scaling factors sx,sy:\n");
scanf("%f%f",&sx,&sy);
setcolor(5);
line(x1,y1,x2,y2);
outtextxy(x2+2,y2+2,"Original line");
x3=x1*sx;
y3=y1*sy;
x4=x2*sx;
y4=y2*sy;
setcolor(7);
```

```
line(x3,y3,x4,y4);
outtextxy(x3+2,y3+2,"Line after scaling");
getch();
}
```

Program for Rotation:

```
#include<stdio.h>
#include<conio.h>
#include<graphics.h>
#include<math.h>
void main()
{
int gd=DETECT,gm;
float x1,y1,x2,y2,x3,y3,x4,y4,a,t;
initgraph(&gd,&gm,"C:\\TurboC3\\BGI");
printf("Enter coordinates of starting point:\n");
scanf("%f%f",&x1,&y1);
printf("Enter coordinates of ending point\n");
scanf("%f%f",&x2,&y2);
printf("Enter angle for rotation\n");
scanf("%f",&a);
setcolor(5);
line(x1,y1,x2,y2);
outtextxy(x2+2,y2+2,"Original line");
t=a*(3.14/180);
x3=(x1*cos(t))-(y1*sin(t));
y3=(x1*sin(t))+(y1*cos(t));
x4=(x2*cos(t))-(y2*sin(t));
y4=(x2*sin(t))+(y2*cos(t));
setcolor(7);
line(x3,y3,x4,y4);
outtextxy(x3+2,y3+2,"Line after rotation");
getch();
}
```

Program for reflection along x-axis:

```
# include <stdio.h>
# include <conio.h>
# include <graphics.h>
# include <math.h>
char IncFlag;
int PolygonPoints[3][2] ={{10,100},{110,100},{110,200}};
void PolyLine()
{
```

```
int iCnt;
cleardevice();
line(0,240,640,240);
line(320,0,320,480);
for (iCnt=0; iCnt<3; iCnt++)
{
line(PolygonPoints[iCnt][0],PolygonPoints[iCnt][1],
PolygonPoints[(iCnt+1)%3][0],PolygonPoints[(iCnt+1)%3][1]);
}
}
void Reflect()
{
float Angle;
int iCnt;
int Tx,Ty;
printf("endl");;
for (iCnt=0; iCnt<3; iCnt++)
PolygonPoints[iCnt][1] = (480 - PolygonPoints[iCnt][1]);
}
void main()
{
int gDriver = DETECT, gMode;
int iCnt;
initgraph(&gDriver, &gMode, "C:\\TurboC3\\BGI");
for (iCnt=0; iCnt<3; iCnt++)
{
PolygonPoints[iCnt][0] += 320;
PolygonPoints[iCnt][1] = 240 - PolygonPoints[iCnt][1];
}
PolyLine();
getch();
Reflect();
PolyLine();
getch();
}
```

Program for Reflection about y-axis:

```
# include <stdio.h>
# include <conio.h>
# include <graphics.h>
# include <math.h>
char IncFlag;
int PolygonPoints[3][2] =
```

```
{{10,100},{110,100},{110,200}};
void PolyLine()
{
int iCnt;
cleardevice();
line(0,240,640,240);
line(320,0,320,480);
for (iCnt=0; iCnt<3; iCnt++)
{
line(PolygonPoints[iCnt][0],PolygonPoints[iCnt][1],
PolygonPoints[(iCnt+1)%3][0],PolygonPoints[(iCnt+1)%3][1]);
}
}
void Reflect()
{
float Angle;
int iCnt;
int Tx,Ty;
for (iCnt=0; iCnt<3; iCnt++)
PolygonPoints[iCnt][0] = (640 - PolygonPoints[iCnt][0]);
}
void main()
{
int gd = DETECT, gm;
int iCnt;
initgraph(&gd, &gm, "C:\\TurboC3\\BGI");
for (iCnt=0; iCnt<3; iCnt++)
{
PolygonPoints[iCnt][0] += 320;
PolygonPoints[iCnt][1] = 240 - PolygonPoints[iCnt][1];
}
PolyLine();
getch();
Reflect();
PolyLine();
getch();
}
```

34

Program for X-shear:

```
#include<stdio.h>
#include<conio.h>
#include<dos.h>
```

```
#include<graphics.h>
void main()
{
int gd=DETECT,gm;
float shx,shy;
initgraph(&gd,&gm,"C:\\TurboC3\\BGI");
printf("Enter shear factor shx along x-axis :");
scanf("%f",&shx);
line(100,0,200,0);
line(200,0,200,200);
line(200,200,100,200);
line(100,200,100,0);
printf("X-shear");
setcolor(12);
line((100+(0*shx)),0,(200+(0*shx)),0);
line((200+(0*shx)),0,(200+(200*shx)),200);
line((200+(200*shx)),200,(100+(200*shx)),200);
line((100+(200*shx)),200,(100+(0*shx)),0);
getch();
}
```

Program for Y-shear:

```
#include<stdio.h>
#include<conio.h>
#include<dos.h>
#include<graphics.h>
void main()
{
int gd=DETECT,gm;
float shx,shy;
initgraph(&gd,&gm,"C:\\TurboC3\\BGI");
printf("Enter shear factor shy along y-axis :");
scanf("%f",&shy);
line(100,10,200,10);
line(200,10,200,200);
line(200,200,100,200);
line(100,200,100,10);
printf("Y-shear");
setcolor(12);
line(100,10+(shy*100),200,10+(shy*200));
line(200,10+(shy*200),200,200+(shy*200));
line(200,200+(shy*200),100,200+(shy*100));
line(100,200+(shy*100),100,10+(shy*100));
```

```
getch();
closegraph();
}
```